CGI PROJECT 1

Students/Group:

- Bruno Ramos 52886
- Lipy Cardoso 63542

Project Description:

- This assignment focuses on building **a WebGL2.0** program for drawing various families of parametric curves.
- Each of the curves parameters can be **freely adjusted via user interface** (mouse and keyboard) in real time.

Feature List:

a) Rendering of multiple parametric curve families selected via keyboard presses corresponding to **keys 1 through 6**;
b) Real time manipulation of any of the curves parametric **coefficients**, either incrementation or decrementation, affecting the curve display instantly;
c) Ability to reset the currently selected curve family's coefficients by pressing the ***"r"*** key;
d) Ability to drag any selected curve family across the browser window using **the mouse**;
e) Ability to zoom in and out using **the mouse wheel**;
    - (Both preserve shape and coherency throughout the movement)
f) Persistent user selection of any given curve coefficient and the ability to modify it;
    - (**Left** and **Right arrows** will select which coefficient to focus)
    - (**Up** and **Down arrows** will respectively increase or decrease its value)
g) Real time auto-animation, either incrementally or decrementally modifying the currently selected curve's parameter. Toggled by pressing **"space bar"**;
    - (When the **Up** or **Down arrow** key is pressed *during auto-animate*, the animation stops and the direction of that key press is **stored.** Upon reactivating the animation, it continues in the previously selected direction)
h) The parameter *t* starts at a fixed lower limit of 0, but the user can increase or decrease the upper limit by pressing **Page Up** or **Page Down**.
i) Curves may be visualized either as a continuous sequence of line segments or as discrete points, each rendered with a size of **5.0 pixels**. The ***'p'*** key allows toggling between these two visualization modes.
j) The application window can be resized at any time, and the canvas dynamically scales to fill the full browser viewport.
k) All the UI overlays (excluding the info panel) can be toggled on or off by pressing the **"h" key**;

Technical Specifications:

- The curve points are **evaluated within** the vertex shader. Up to a *maximum* of **60,000** points can be used to render the curve.
- The vertex shader also receives, as uniform variables, the coefficient values for the selected curve, an identifier for the curve family, and the parameter limits t_0 and t_1.
- The application correctly implements **aspect ratio preservation** to maintain the true proportions of all shapes, ensuring the curve is not visually distorted.

Extra Features:

a) A **DVD-logo style animation** activates on program startup (Curve 0); the curve bounces within the viewport and changes color upon each collision. The last color set will carry over as default for drawing the parametric curves.
b) The **Single-color picker** allows the user to select and apply a uniform color to the entire curve.
c) The **Gradient color picker** enables gradual color transitions along the curve by defining start and end colors.
    - (Starting Color full intensity in point 0, End Color full intensity in final point)
d) An **Animation speed slider** that provides real-time control over the speed of the automatic coefficient animation in steps of 0.01 units.
e) The **Coefficient visual selector** highlights the currently active coefficient in the information panel using **“**coef**” notation** for clarity.

Notes on A.I. use:

ChatGPT/Copilot were used throughout the project as a helper for faster auto-complete, fixing syntax issues, and rubber-ducking solutions when things got stuck.
It also helped explain some WebGL concepts and facilitated the access to documentation.
Even though the AI provided sporadic guidance, all the coding, problem-solving, and final implementation were done by the group’s students following a branch-based development cycle with periodic reviews and meetings.

GLSL Program Inputs

(VERTEX SHADER)

Attributes:

- `a_position` (uint)
  Sequence index of each sample point along the curve. Used to compute the parameter `t` and the normalized value `v_t` for gradient coloring.
  The buffer is filled on the CPU with a `Uint32Array` of up to 60,000 points and connected via `gl.vertexAttribIPointer`.

Uniforms:

- `u_curveFamily` (int)
  Selects the curve family to render. Default is 0; controlled via keyboard keys 1–6.

- `u_aspect` (float)
  Aspect ratio of the canvas, used to preserve the correct shape of curves. Updated automatically on window resize.

- `u_samplePoints` (float)
  Total number of points sampled along the curve. Default is 60,000; adjustable with the '+' and '-' keys.

- `u_zoom` (float)
  Zoom factor applied to the curve coordinates. Default is 1.0; adjustable via the mouse wheel.

- `u_pan` (vec2)
  Panning offset of the scene (X, Y). Controlled via mouse dragging or by the DVD-logo animation.

- `u_tLimits` (vec2)
  Lower and upper bounds (t_0 and t_1) of the curve parameter. Default $t_0 = 0$, $t_1 = 2\pi$; adjustable with PageUp and PageDown keys.

- `u_coefficients` (vec3)
  Coefficients affecting the shape of the selected curve. Default `[1.0, 0.0, 1.0]`; modifiable with arrow keys or auto-animation.

Varying:

- `v_t` (float)
  Normalized parameter along the curve, computed as `a_position / u_samplePoints`.
  It ranges from 0 to 1 and is passed to the fragment shader to interpolate between the starting and ending colors when in gradient mode.

(FRAGMENT SHADER)

Uniforms:

- `u_singleColor` (vec4)
  Color used in single-color mode. Default is red `(1,0,0,1)`; selectable via the color picker UI.

- `u_startingColor` (vec4)
  Gradient start color for the curve. Selected via UI color picker.

- `u_endingColor` (vec4)
  Gradient end color for the curve. Selected via UI color picker.

- `u_colorModeFlag` (bool)
  Determines the color mode: `true` = single color, `false` = gradient. Controlled via UI buttons.

Varying:

- `v_t` (float)
  Interpolation factor for gradient mode.

Output:

- `frag_color` (vec4)
  The final color of the fragment. Determined either by `u_singleColor` or by linearly interpolating between `u_startingColor` and `u_endingColor` using `v_t`.